Ex 4-1

1/30/24 07:59

Exercises

Exercise 4.1

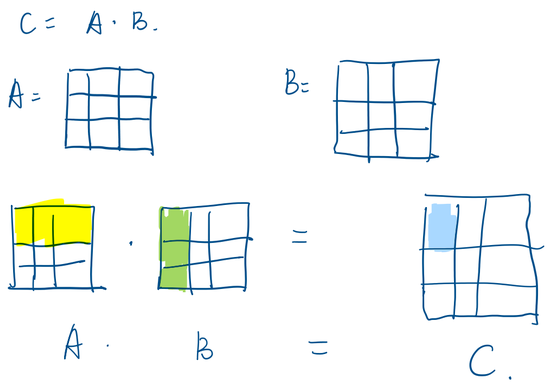
The aim of this exercise is to learn the basics of using the BLAS library.

Recall that full documentation for the BLAS interface can be found at http://www.netlib.org/blas and a quick reference guide to the available routines is available to download from Blackboard Learn.

1. Create a new program which allocates an array of N double-precision floating-point numbers, x_i, and populates them with pseudo-random numbers between 0 and 1. Choose N small to start with.
2. Using BLAS routines, compute the three quantities

$$d = \sqrt{\left(\sum_{i=1}^{N} x_i^2\right)} \quad \text{and} \quad s = \sum_{i=1}^{N} |x_i| \quad \text{and} \quad m = \max_i |x_i|$$

3. Write a function which manually computes these quantities and verify you have used the BLAS correctly.



C { row: i, column: j } A { row: i, column: k. } B { row: j, column: k.

$C_{11} = A_{11} \cdot B_{11} + A_{12} \cdot B_{21} + A_{13} \cdot B_{31}$.

C[i·n+j]

eg. $C_{11} \Rightarrow C_{00} = 0 \cdot 3 + 0 = 0$.

$A_{11} \Rightarrow A_{00} = A[i \cdot n + k]$. ✓

$B_{11} \Rightarrow B_{00} = B[k \cdot n + j]$. ✓

```cpp
#include <iostream>
#include <cmath>
using namespace std;
// Directly calling Fortran API equally valid
#include "cblas.h"
double ComputeNorm(const int n, const double* x) {
    double result = 0.0;
    for (int i = 0; i < n; ++i) {
        result += x[i]*x[i];
    }
    return sqrt(result);
}
double ComputeSum(const int n, const double* x) {
    double result = 0.0;
    for (int i = 0; i < n; ++i) {
        result += x[i];
    }
    return result;
}
int ComputeMaxIndex(const int n, const double* x) {
    double max = 0.0;    // Assume only positive numbers in x
    int max_idx = 0;
    for (int i = 0; i < n; ++i) {
        if (x[i] > max) {
            max = x[i];
            max_idx = i;
        }
    }
    return max_idx;
}
int main() {
    // Declare the array
    const int n = 100;
    double* x = new double[n];
    // Seed the random number generate with the current epoch time
    srand(time(0));
    // Generate random numbers and print them to the screen
    for (int i = 0; i < n; ++i) {
        x[i] = (double)rand()/RAND_MAX;
        cout << x[i] << endl;
    }
    // Use BLAS to generate L2-norm
    double norm = cblas_dnrm2(n, x, 1);
    // Since x[i] >= 0, we can use DASUM to compute the sum
    double sum  = cblas_dasum(n, x, 1);
    // Use the BLAS routine IDAMAX to find the index of entry with maximum value
    int    max  = cblas_idamax(n, x, 1);
    // Print out the values
    cout << "Norm:      " << norm << endl;
    cout << "Sum:       " << sum << endl;
    cout << "Max index: " << max << endl;
    cout << "Max value: " << x[max] << endl;
    // Manual verification
    cout << "Verify norm:      " << ComputeNorm(n, x) << endl;
    cout << "Verify sum:       " << ComputeSum(n, x) << endl;
    int man_max = ComputeMaxIndex(n, x);
    cout << "Verify max idx:   " << man_max << endl;
    cout << "Verify max value: " << x[man_max] << endl;
    return 0;
}
```